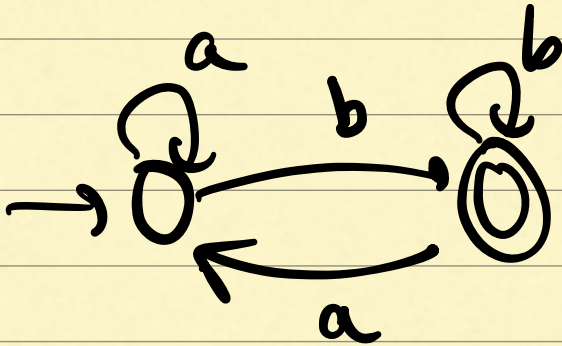


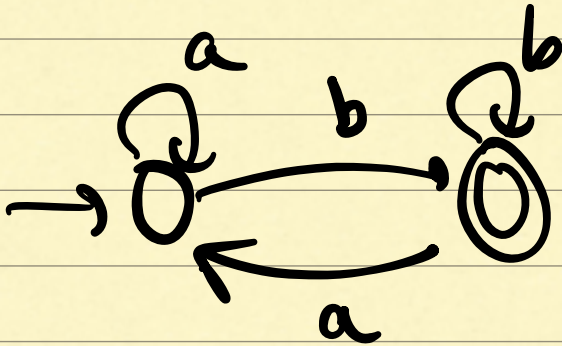
Recap from

CS 228: Logic in Computer Science

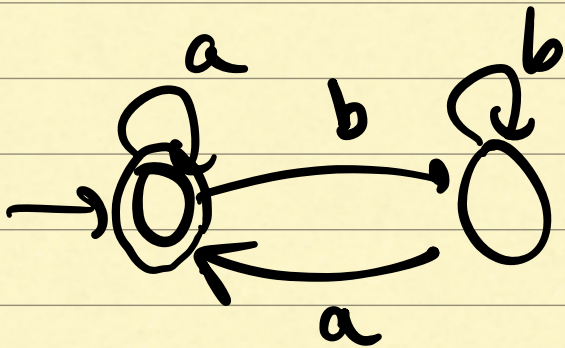
a DFA for the language consisting of all words over $\Sigma = \{a, b\}$ that ends with b.



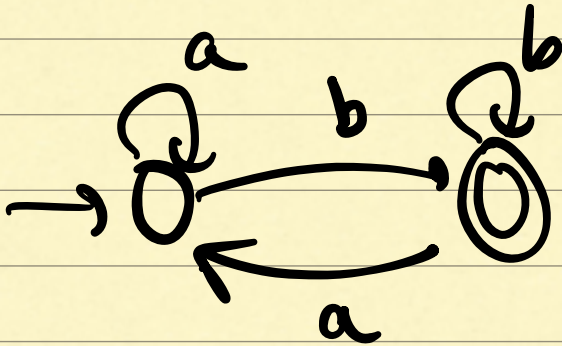
a **DFA** for the language consisting of all words over $\Sigma = \{a, b\}$ that ends with b .



Q: what is the complement of this language?

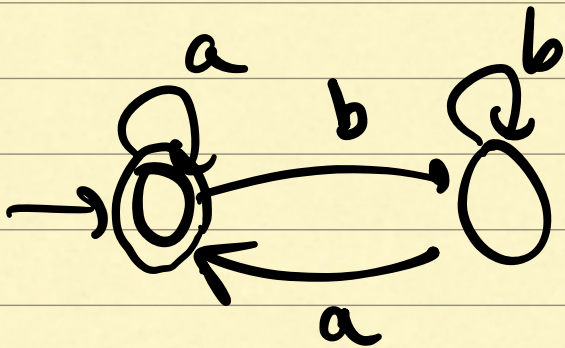


a **DFA** for the language consisting of all words over $\Sigma = \{a, b\}$ that ends with b .

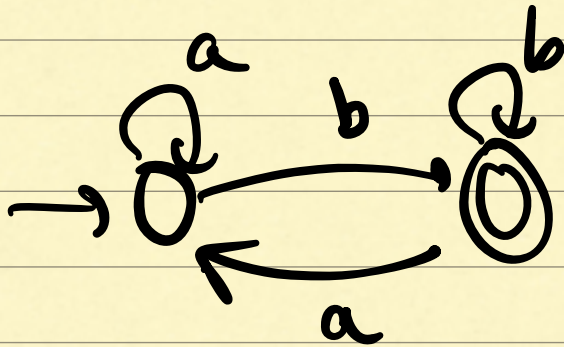


Q: what is the complement of this language?

Ans: all words ending with a ?

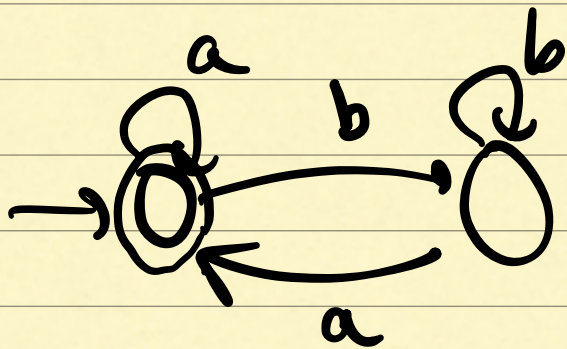


a **DFA** for the language consisting of all words over $\Sigma = \{a, b\}$ that ends with b .



Q: what is the complement of this language?

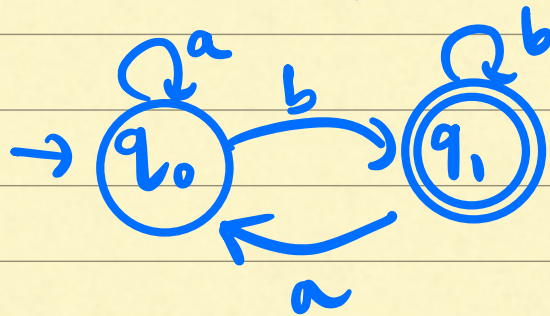
Ans: all words ending with a ?



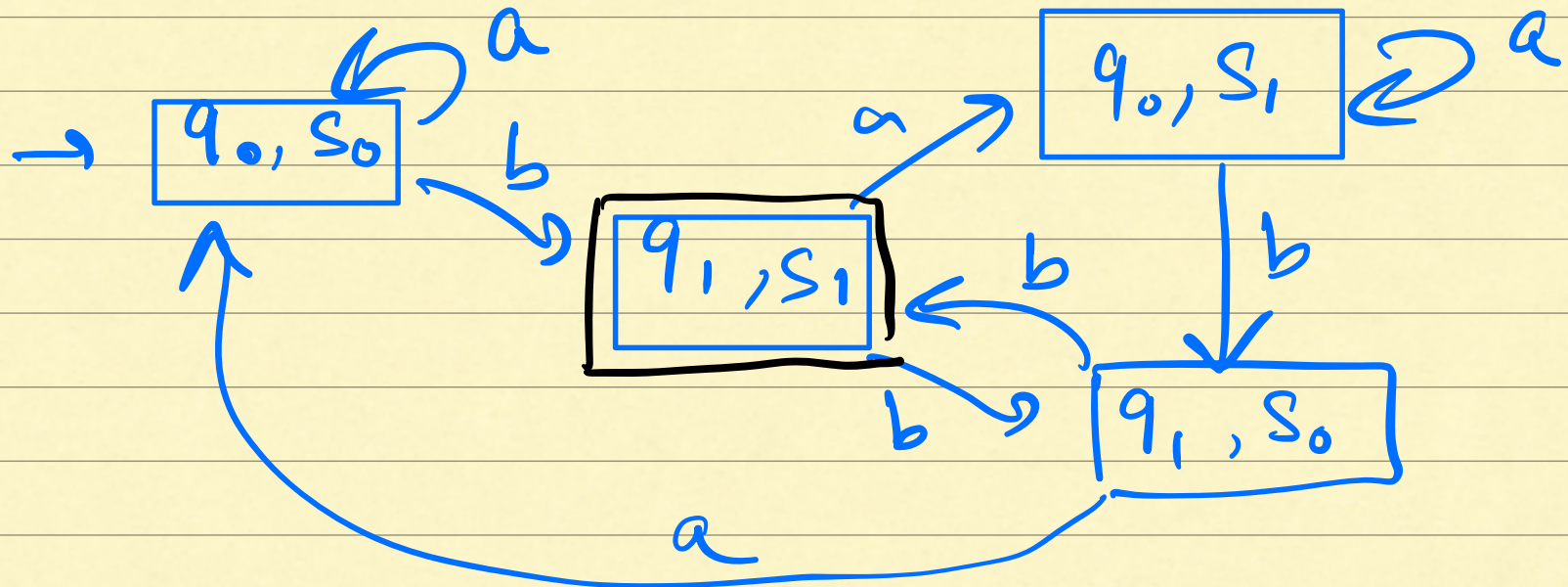
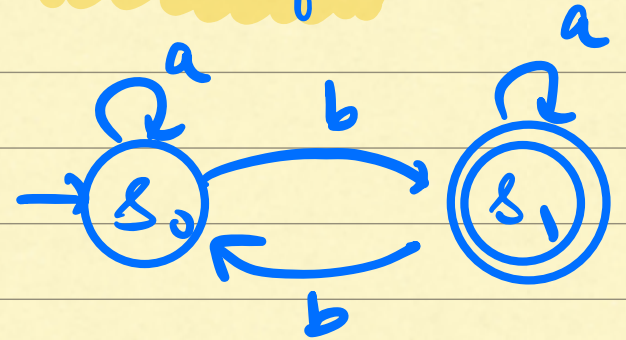
what about ϵ ? $\epsilon \in \bar{L}$?

What about the intersection of two regular languages?

words ending with b & words with odd no. of b



$\cap$



Formally

$$A_1 = (Q_1, \Sigma, \delta_1, q_0, F_1) \quad A_2 = (Q_2, \Sigma, \delta_2, s_0, F_2)$$

$$A = (Q_1 \times Q_2, \Sigma, \delta, (q_0, s_0), F_1 \times F_2), \text{ where}$$

Claim: $L(A) = L(A_1) \cap L(A_2)$

$$\delta((q, s), a) = (\delta_1(q, a), \delta_2(s, a))$$

Proof: $w \in L(A)$

$$\Leftrightarrow \hat{\delta}((q_0, s_0), w) \in F_1 \times F_2$$

$$\Leftrightarrow (\hat{\delta}_1(q_0, w), \hat{\delta}_2(s_0, w)) \in F_1 \times F_2$$

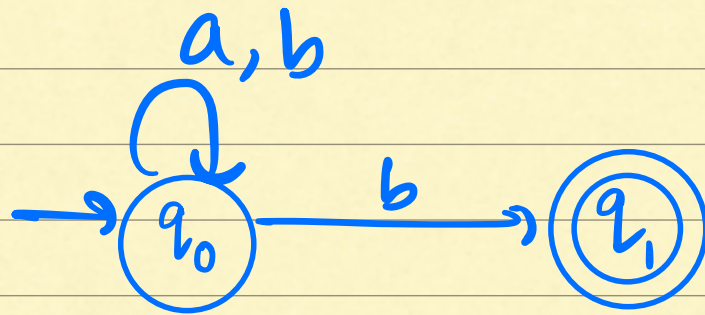
$$\Leftrightarrow \hat{\delta}_1(q_0, w) \in F_1 \text{ \& \& } \hat{\delta}_2(s_0, w) \in F_2$$

$$\Leftrightarrow w \in L(A_1) \text{ \& \& } w \in L(A_2)$$

$$\Leftrightarrow w \in L(A_1) \cap L(A_2)$$

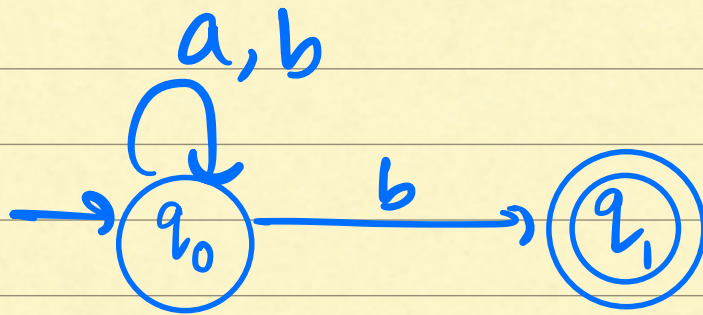
What happens if we allow more than one outgoing transition from a state on the same letter?

consider the following automaton:



- not all transitions are specified
- more than one transition on a letter

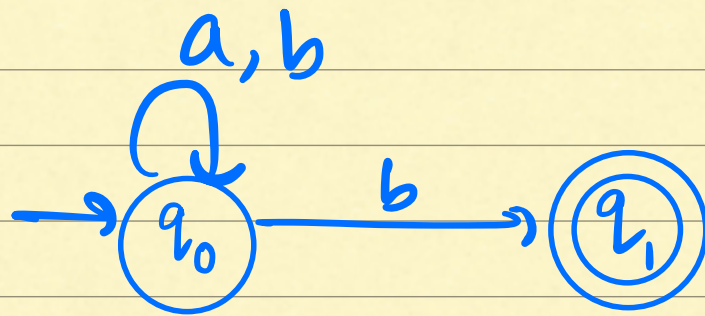
consider the following automaton:



What is the **run** of this automaton on

a b a

consider the following automaton:

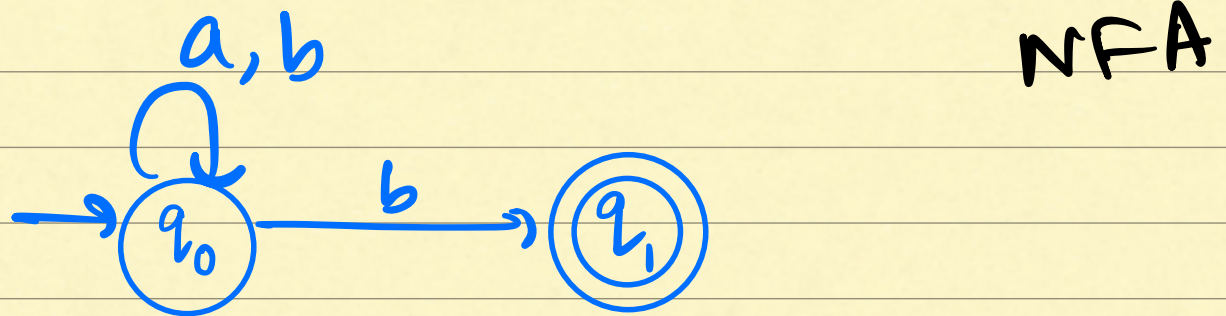


What is the **run** of this automaton on

a b a

multiple runs exist !

consider the following automaton:

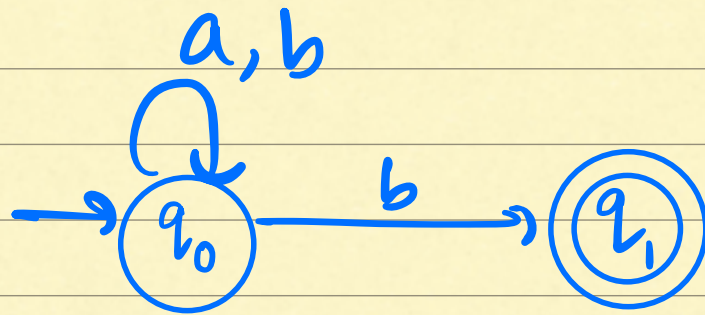


What is the run of this automaton on
a b a

multiple runs exist!

non-determinism

consider the following automaton:

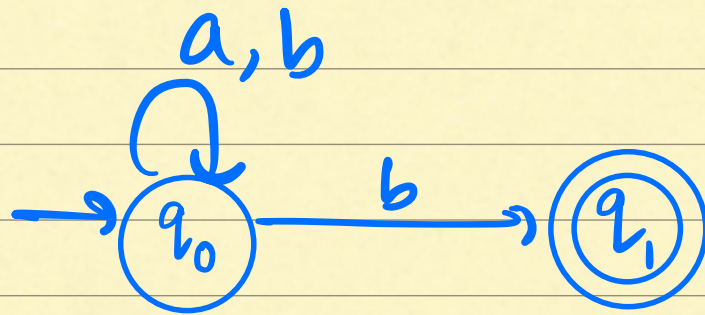


Formally, an **NFA** is a tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where $\delta: Q \times \Sigma \rightarrow 2^Q$

consider the following automaton:



Formally, an **NFA** is a tuple

$$(Q, \Sigma, \delta, q_0, F)$$

$$\text{where } \delta: Q \times \Sigma \rightarrow 2^Q$$

Acceptance condition: at least one accepting run

language of the above automaton: all words ending with b

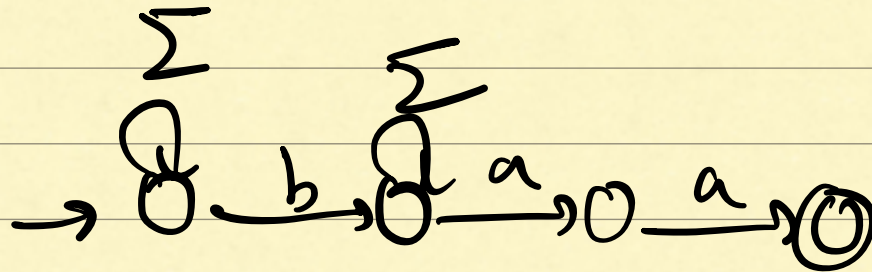
Is it Regular?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Even length words
- ▶ Contains *abb*
- ▶ Right before the last position is an *a*

- ▶ Contains a *b* and ends with *aa*
- ▶ Begins with *a*, ends with *b*, and has a pair of consecutive *a*'s
- ▶ There are two occurrences of *b* between which only *a*'s occur

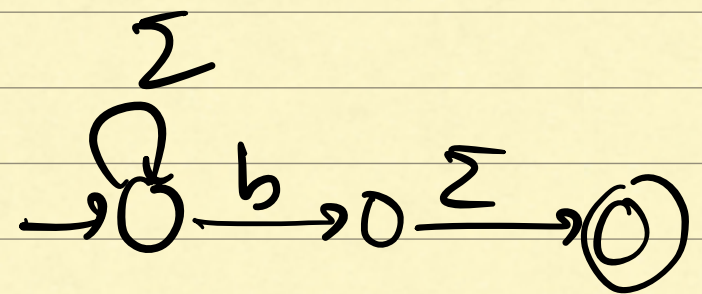
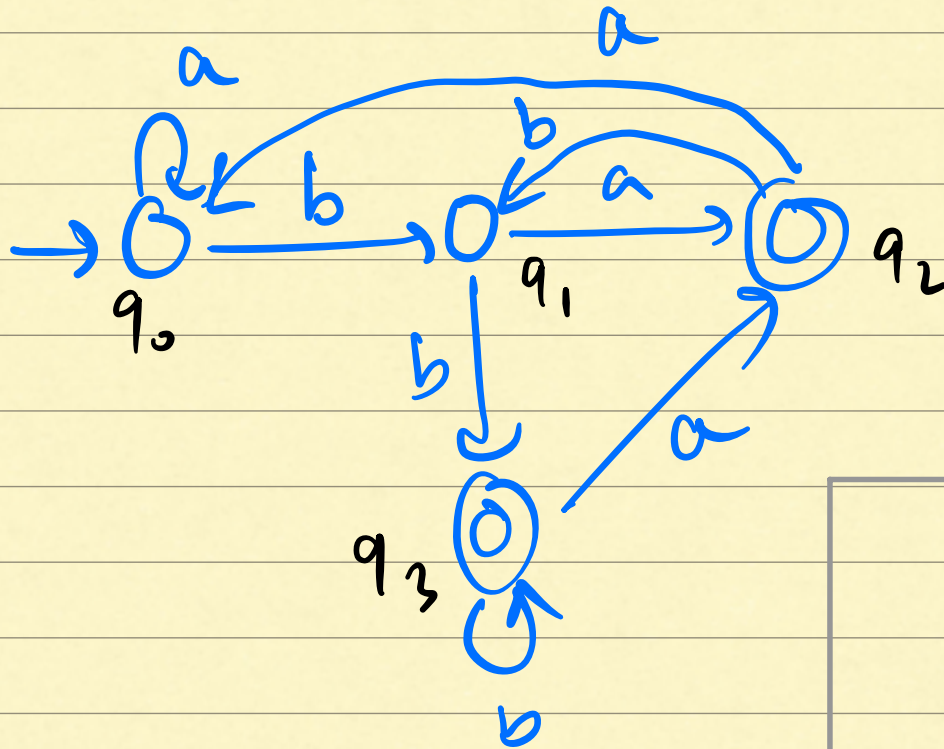
accept words containing b and ending with aa



$L = \{ w \in \{0,1\}^* \mid \text{the second last letter of } w \text{ is } b \}$

$L = \{ w \in \{0,1\}^* \mid \text{the second last letter of } w \text{ is } b \}$

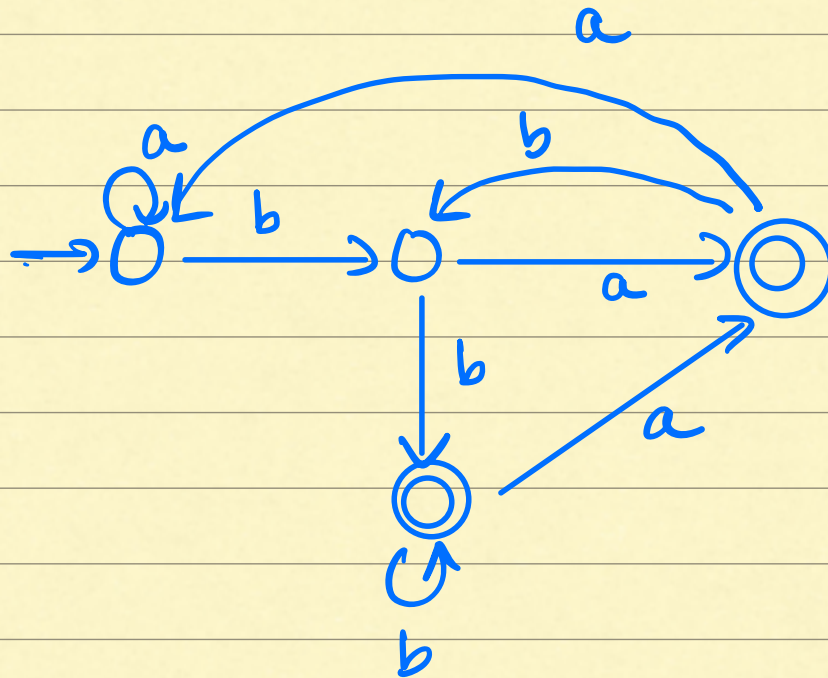
can we draw a DFA for this language?



NFA

$L = \{ w \in \{0,1\}^* \mid \text{the second last letter of } w \text{ is } b \}$

can we draw a DFA for this language?



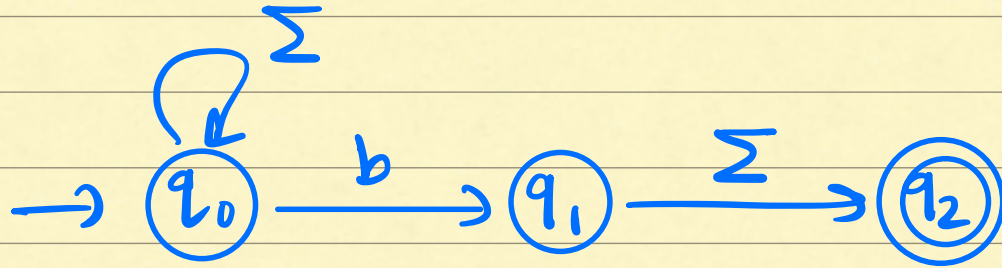
Q: Can you draw a DFA with ≤ 3 states?

Are NFA more powerful than DFA?

... in other words ...

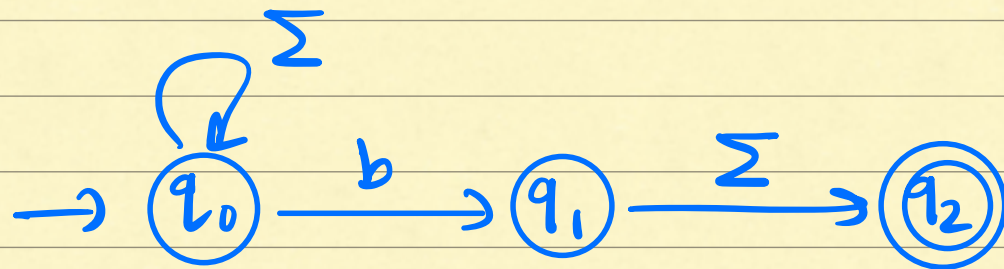
?
 $\exists$ a language for which there exists an NFA
but no DFA?

Second last letter is b

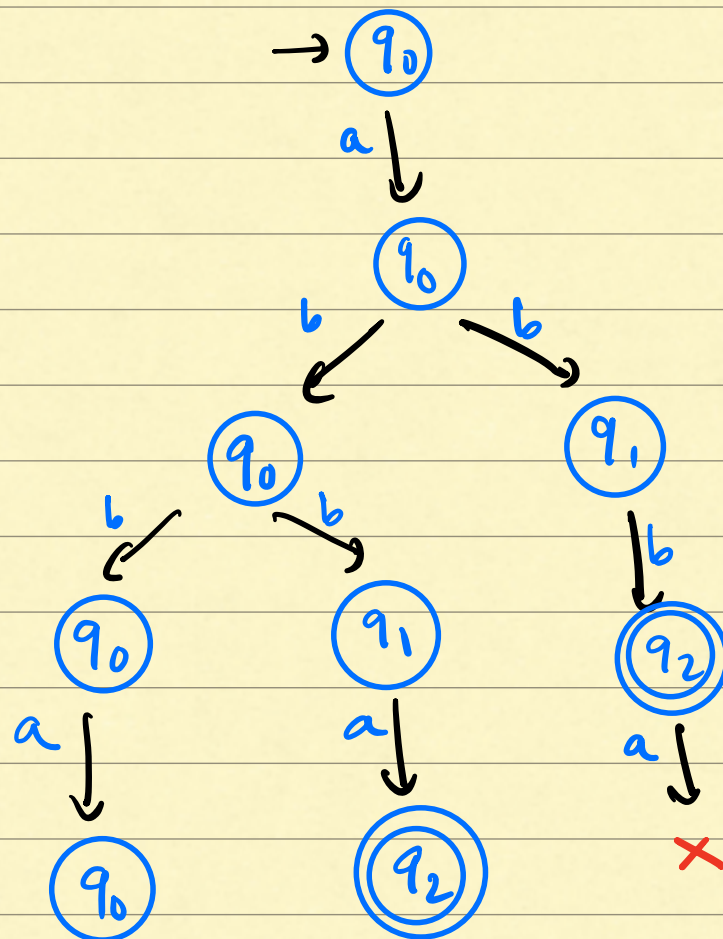


consider the word
a a b a

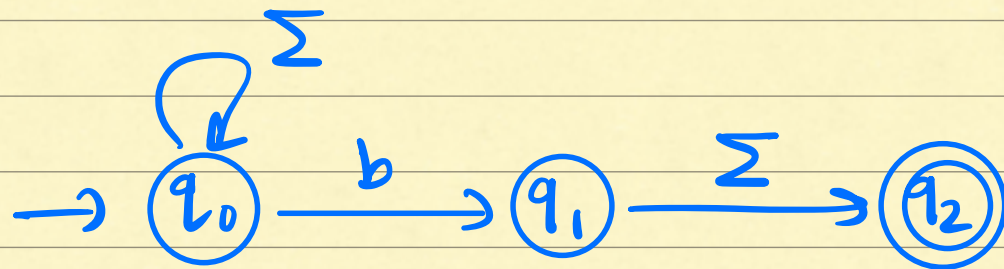
Second last letter is b



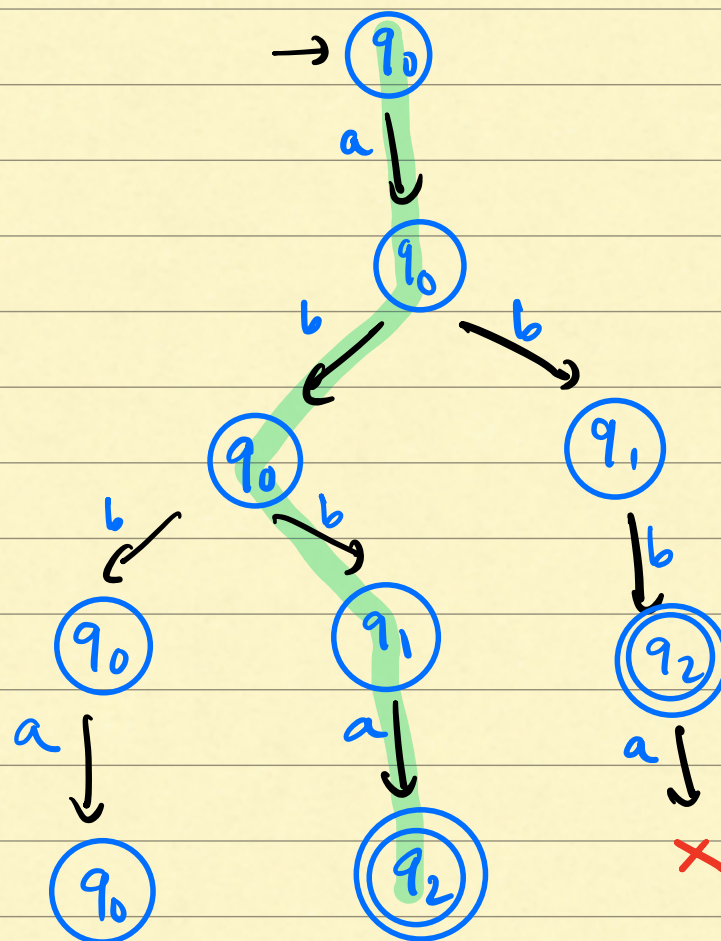
consider the word
a b b a



Second last letter is b



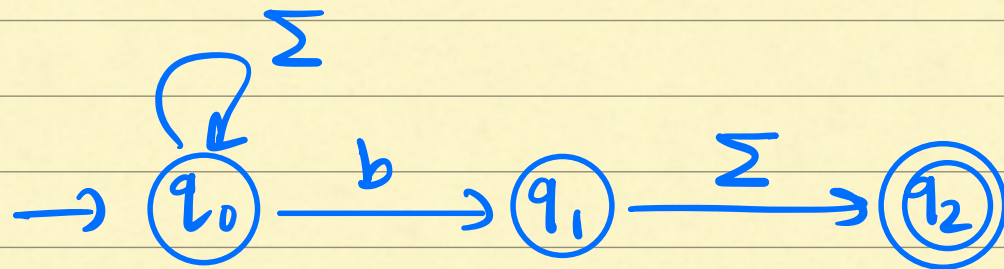
consider the word
a b b a



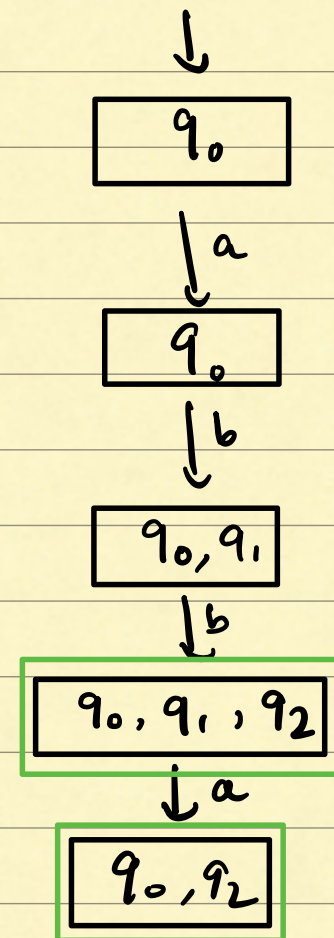
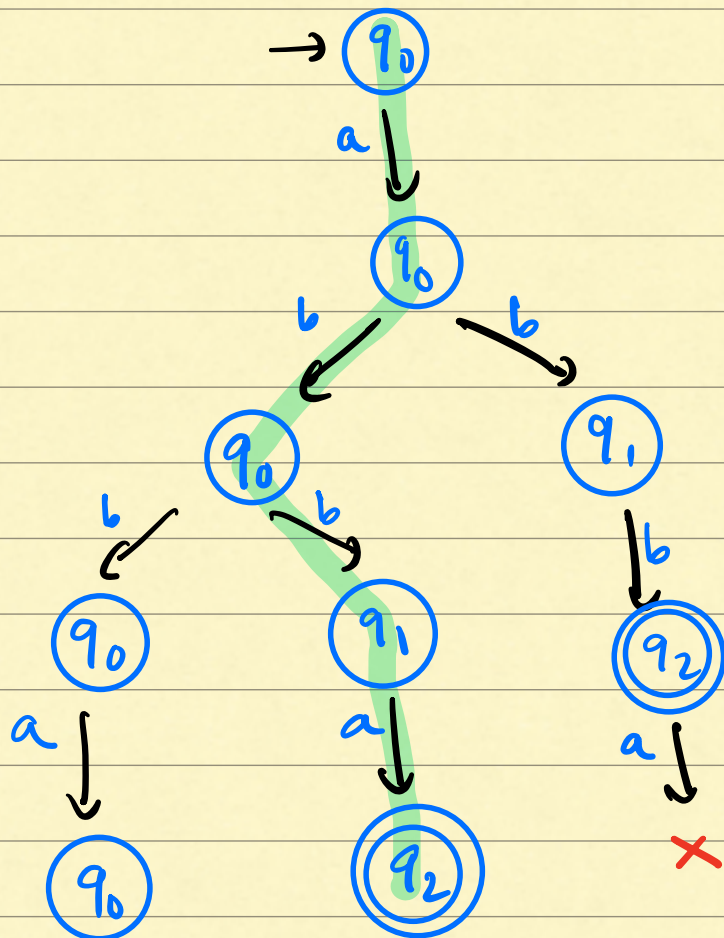
one accepting path

✓ automaton accepts
the word
abba

Second last letter is b



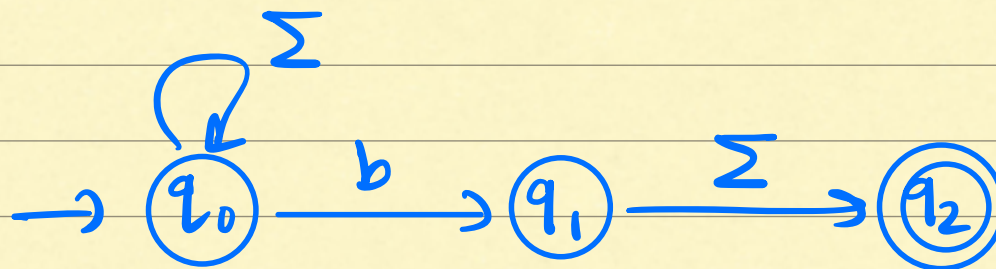
consider the word
a b b a



NFA $\longrightarrow$ DFA

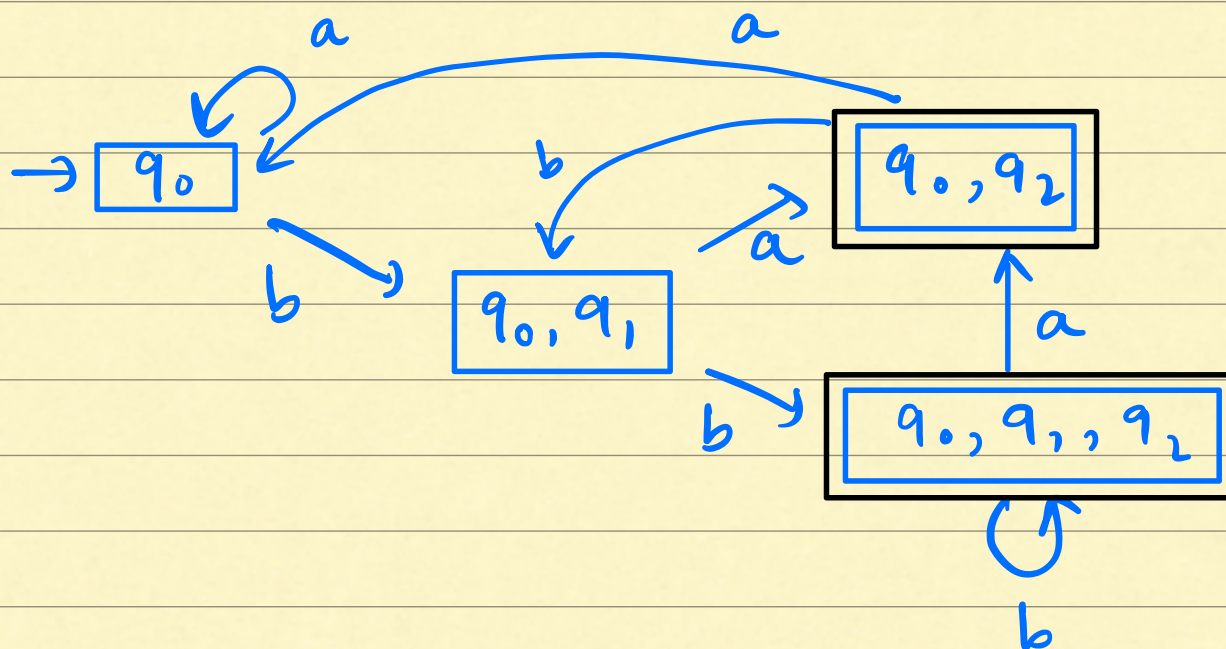
Subset construction

NFA



language
equivalent

DFA



NFA $A_N = (Q, \Sigma, \delta, q_0, F)$

DFA $A_D = (2^Q, \Sigma, \Delta, \{q_0\}, F_N)$

where $\Delta: 2^Q \times \Sigma \rightarrow 2^Q$ defined as

$$\Delta(\underline{q}, a) = \bigcup_{q \in \underline{q}} \{\delta(q, a)\} \quad \left| \begin{array}{l} \text{here } \underline{q} \subseteq Q \\ \text{i.e. } \underline{q} \in 2^Q \end{array} \right.$$

and $F_N \subseteq 2^Q$, $F_N = \{\underline{q} \mid \underline{q} \cap F \neq \emptyset\}$

Claim: $L(A_D) = L(A_N)$

Proof: $w \in L(A_D) \iff \hat{\Delta}(\{q_0\}, w) \in F_N$

$$w \in L(A_N) \iff \hat{\delta}(q_0, w) \cap F \neq \emptyset \iff \hat{\Delta}(\{q_0\}, w) \cap F \neq \emptyset$$

$\Downarrow$

Every NFA can be converted into a DFA

... therefore ...

A language is regular iff there exists an NFA
for the language

What if we modify the accepting condition

accept a word

only if

all the runs end in final states?

Does it still recognize only regular languages?

DFA States

- ▶ Each state is a **bucket** holding infinitely many words
- ▶ Thus we have good and bad buckets
- ▶ The buckets partition Σ^*
- ▶ **Good buckets** determine the language accepted by the DFA
- ▶ Words that land in bad buckets are not accepted by the DFA

DFA States

- ▶ Each state is a **bucket** holding infinitely many words
- ▶ Thus we have good and bad buckets
- ▶ The buckets partition Σ^*
- ▶ **Good buckets** determine the language accepted by the DFA
- ▶ Words that land in bad buckets are not accepted by the DFA

For NFA states, these do not hold
in general!